

Advanced Lesson 9 - Exercises: Defaults, Definition Order, Tags and Attributes

Mikey Ferguson

MATH 7013 - Advanced Mathematical Modeling

1: Optional arguments with default values

```
Clear[f, x, y, z];  
f[x_, y_ : 10, z_ : 20] := {x, y, z}  
?f
```

```
{f[1, 2, 3, 4], f[1, 2, 3], f[1, 2], f[1], f[]}
```

The colon in a pattern supplies a default: `y_ : 10` means "an argument here, or 10 if none is given".

`f[1, 2, 3, 4]` returns unevaluated - four arguments do not match a pattern that accepts at most three.

`f[1, 2, 3]` returns `{1, 2, 3}`; every slot was filled explicitly.

`f[1, 2]` returns `{1, 2, 20}`; `z` fell back to its default.

`f[1]` returns `{1, 10, 20}`; both optional slots defaulted.

`f[]` returns unevaluated - `x` has no default, so a call with no arguments cannot match.

Defaults fill from the right. There is no way to supply `z` while omitting `y`, which is exactly why optional arguments are ordered least-important-last.

2: Definition order - anything, odd, even, and the literal 2

```
Clear[f, i];
f[x_] := "x is anything";
f[x_?OddQ] := "x is odd";
f[x_?EvenQ] := "x is even";
f[2] := "x = 2"

?f
```

```
TableForm[Table[{i, f[i]}, {i, 1, 4,  $\frac{1}{2}}$ }]
```

Version A, as printed. This is the case where the naive prediction is wrong, so it is worth reading "?f" carefully. The definitions were entered with the catch-all f[x_] first, but the stored order is f[2], then f[x_?OddQ], then f[x_?EvenQ], and only then f[x_]. Mathematica reordered them.

It does this because it can rank these particular patterns by specificity: a literal argument beats a pattern, and a pattern carrying a PatternTest beats a bare Blank. So the catch-all is demoted to last no matter where it is typed, and the table comes out correct - 1 and 3 are odd, 4 is even, 2 is "x = 2", and the half-integers, which are neither odd nor even, fall through to "x is anything".

```
Clear[f, i];
f[x_?OddQ] := "x is odd";
f[x_] := "x is anything";
f[x_?EvenQ] := "x is even";
f[2] := "x = 2"

?f
```

```
TableForm[Table[{i, f[i]}, {i, 1, 4,  $\frac{1}{2}}$ }]
```

Version B, with the second and third lines interchanged. "?f" reports exactly the same stored order as Version A, and the table is identical. Interchanging those two lines changed nothing at all, because the sort had already put them in that order.

Which code segment would work if the goal is to identify numbers as even, odd or equal to 2? Both of them. That is the answer, and it is not the answer the exercise's phrasing invites. The reordering is doing the work, not the typing. The version below is written in the order Mathematica will use anyway, which is worth doing for the reader's sake even though the kernel does not need it:

```

Clear[f, i];
f[2] := "x = 2";
f[x_?OddQ] := "x is odd";
f[x_?EvenQ] := "x is even";
f[x_] := "x is anything"

TableForm[Table[{i, f[i]}, {i, 1, 4,  $\frac{1}{2}}$ }]

```

Same table again, as expected.

The rule to take away is narrower than "definition order decides everything".

Mathematica reorders definitions it can rank, and a bare Blank always loses to a literal and to a PatternTest. Where order genuinely does decide the answer is when two definitions are equally specific and Mathematica has no grounds to prefer either - which is exactly the situation in Exercise 3.

3: Four tests and a catch-all, and why the order changes the answers

```

Clear[f, i];
f[x_?OddQ] := " x is odd";
f[x_?Positive] := " x is positive";
f[x_?IntegerQ] := " x is an integer";
f[2] := " x = 2";
f[x_] := " x is arbitrary"

?f

TableForm[Table[{i, f[i]}, {i, -2, 4,  $\frac{1}{2}}$ }]

```

Version A. "?f" shows f[2] hoisted to the front and the four pattern rules kept in entry order. Reading the table: -2 is "an integer" (it is not odd and not positive, so the third rule catches it), -1 is "odd", the negative half-integers are "arbitrary", 0 is "an integer", the positive half-integers are "positive", the positive odd integers are "odd" because that rule is first, 2 is "x = 2", and 4 is "positive" because Positive is tested before IntegerQ.

```

Clear[f, i];
f[x_?OddQ] := " x is odd";
f[x_?IntegerQ] := " x is an integer";
f[x_?Positive] := " x is positive";
f[2] := " x = 2";
f[x_] := " x is arbitrary"

```

```
TableForm[Table[{i, f[i]}, {i, -2, 4,  $\frac{1}{2}}$ }]
```

Version B, with the second and third lines interchanged. The results differ for every positive non-odd integer: 4 is now "an integer" rather than "positive", and 0 is unchanged only because it was already reaching the integer rule.

Why the results differ, and why this exercise behaves differently from Exercise 2. All three of `x_?OddQ`, `x_?Positive` and `x_?IntegerQ` have the same shape - a Blank wrapped in a PatternTest - so Mathematica can rank them against the literal `f[2]` and against the bare catch-all `f[x_]`, but it cannot rank them against each other. Among equally specific definitions it preserves entry order.

The tests are also not mutually exclusive: 4 satisfies both `IntegerQ` and `Positive`. So whichever of those two was typed first wins, and interchanging the lines changes the answer. Overlapping conditions of equal specificity are the case where definition order is part of the meaning of the program rather than a detail of its layout. The only way to make order irrelevant is to make the conditions disjoint.

4: Teaching Integrate about a new function, by tagging

```
ClearAll[myFunction, myFunctionIntegral, x, y];
```

```
 $\int$ myFunction[x_] dx_ := myFunctionIntegral[x]
```

Part 1, and what happens. Mathematica refuses, with `SetDelayed::write - "Tag Integrate in Integrate[myFunction[x_], x_] is Protected"`. A definition is filed under the head of its left-hand side, which here is `Integrate`, a protected built-in symbol. Adding to it would mean modifying the system function, and that is exactly what protection prevents.

```
ClearAll[myFunction];
```

```
myFunction /:  $\int$ myFunction[x_] dx_ := myFunctionIntegral[x];
```

Part 2. `TagSetDelayed` - written with `"/:"` - files the definition under `myFunction` instead of under `Integrate`. That symbol belongs to the user, so no protection is violated. Definitions stored this way are called upvalues: Mathematica consults them when the symbol appears among the arguments of some other function, which is exactly the situation here.

```
??myFunction
```

Part 3. "??myFunction" reports the rule under the heading of upvalues, not downvalues - the distinction being where the definition is filed, and hence when it is looked up.

$$\int \sin[x] \, dx$$

$$\int \text{myFunction}[x] \, dx$$

$$\int \text{myFunction}[y] \, dy$$

$$\text{Head}\left[\int \text{myFunction}[x] \, dx\right]$$

Parts 4 to 7. The ordinary integral still returns $-\cos[x]$; nothing about Integrate was damaged. The two new integrals return myFunctionIntegral[x] and myFunctionIntegral[y] - the rule is stated in terms of a pattern, so it works for any integration variable, not just the one it was written with. The Head of the result is myFunctionIntegral. That is the point: the expression is no longer an unevaluated Integrate, it has been rewritten into the user's own function, and any further processing sees the new head.

5: An orderless function

```
ClearAll[myFunction, a, b, c, d];
SetAttributes[myFunction, Orderless];

??myFunction

{myFunction[1, 2, 3], myFunction[3, 2, 1]}

{myFunction[a, b, c, d], myFunction[d, c, b, a]}

myFunction[c, b, a, d] == myFunction[d, c, b, a]
```

Orderless tells Mathematica the function is symmetric in its arguments, so it sorts them into canonical order before doing anything else. The two calls in each pair are therefore literally the same expression after evaluation - not merely equal in value, but identical in form.

The comparison in the last cell returns True for that reason: both sides sort to myFunction[a, b, c, d] and the equality test compares identical expressions. Plus and Times carry this attribute, which is why $a + b$ and $b + a$ are the same expression rather than two expressions that happen to agree.

6: Holding only the first argument

```
ClearAll[myFunction, a, b, c, d];
SetAttributes[myFunction, HoldFirst];
```

```
??myFunction
```

```
a = 1;
b = 2;
c = 3;
d = 4;
myFunction[a, b, c, d]
```

Parts 2 and 3. With no downvalues defined, the call returns unevaluated - but not unchanged. It comes back as `myFunction[a, 2, 3, 4]`: the first argument is held as the symbol `a`, while the remaining three were evaluated to their assigned values. That is precisely what `HoldFirst` means, and this cell shows it in isolation before any definition can obscure it.

```
myFunction[a_, b_, c_, d_] := {a, b, c, d}
```

```
??myFunction
```

```
a = 1;
b = 2;
c = 3;
d = 4;
myFunction[a, b, c, d]
```

Parts 4 to 6. The result is `{1, 2, 3, 4}`. The first slot did end up as 1 even though it was held.

Part 7, where `a` was evaluated. Holding stops evaluation of the argument before the function is entered. It does not make the argument permanently inert. The held symbol `a` is bound to the pattern variable `a`, substituted into the right-hand side, and the right-hand side is then evaluated in the ordinary way - at which point `a` is looked up and yields 1. So the evaluation happened after pattern matching and substitution, on the way out, rather than before the call, on the way in. Wrapping the body in `Hold`, or using `HoldForm`, is what would keep it symbolic. Tracing the call shows the substitution and the subsequent lookup as two separate steps.

7: A usage message and full holding

```
ClearAll[myFunction, x, y, z];  
myFunction::usage = "myFunction [x, y, z, ... ] is important to understand.";  
myFunction[x_, y_, z_] := Sin[x y z];  
SetAttributes[myFunction, HoldAll];  
  
?myFunction  
  
??myFunction
```

A usage message is an ordinary string assigned to the symbol's "usage" message name, and it is what "?" prints. "?" shows the usage message alone; "??" shows the usage message plus the attributes and every stored definition. Documenting a function is therefore not a comment convention - the documentation is data attached to the symbol, retrievable by the same mechanism that retrieves its definitions.